ARQUITECTURA DEL SOFTWARE / UNIDAD II

DEL MODELO AL PROTOTIPO

SIGSU

DOCUMENTO	REPORTE TÉCNICO INTEGRADOR
EVIDENCIA	EVIDENCIA 4 · HETEROEVALUACIÓN 20 %
ESTUDIANTES	ELIAN RAMÍREZ GRAGEDA · JORGE SEBASTIÁN CHAVIRA LÓPEZ DIEGO CERVANTES ARRAS · WALTER JESÚS GONZÁLEZ ZÚÑIGA
CASO ASIGNADO	SIGSU · SOLICITUDES DE SERVICIOS EN LA UTCH
GRUPO	IDGS71N · EQUIPO 3
DOCENTE	ALFONSO BARROSO
FECHA	29 DE OCTUBRE DE 2026
LUGAR	CHIHUAHUA, CHIHUAHUA

UTCH | Universidad Tecnológica de Chihuahua

EVIDENCIA ACADÉMICA
OCTUBRE 2026

Caso y alcance

> En la UTCH los estudiantes piden sus trámites por correo, por mensaje o en formatos sueltos. Servicios Escolares atiende las constancias, Tecnologías de la Información las cuentas y el equipo, y Recursos Materiales el préstamo de aulas y laboratorios. Cada área guarda las peticiones a su manera, así que nadie sabe qué se pidió, quién lo atiende ni cuánto tarda.

Los síntomas tienen tres causas. Sin un registro único, el estudiante insiste por otro canal y se generan duplicados. Sin estados, nadie sabe si un trámite avanza. Sin evidencia, no se pueden medir los tiempos. SIGSU convierte cada trámite en una solicitud con folio, estudiante, servicio, fecha de vencimiento y estado, y registra cada cambio.

Actor	Qué hace en SIGSU
Estudiante	Consulta el catálogo, registra una solicitud con sus datos y adjuntos, consulta su estado e historial, y la cancela mientras nadie la atiende.
Responsable de área	Ve la bandeja de su área ordenada por vencimiento, toma una solicitud, registra observaciones y la cierra como atendida o rechazada.
Administrador	Gestiona usuarios, roles, áreas, servicios, versiones de formulario y días inhábiles.
Proceso de vencimientos	Cada 15 minutos marca las solicitudes cuyo plazo venció y avisa al área.

Tabla 1. Actores de SIGSU.

Requerimientos que guiaron el diseño

Los requerimientos que siguen son los que obligaron a tomar una decisión de arquitectura o de modelo. Los demás, como consultar el catálogo o ver el historial propio, son consultas que cualquier diseño resuelve.

- **Un estudiante no puede tener dos solicitudes abiertas del mismo servicio.** Debe cumplirse aunque las dos peticiones lleguen al mismo tiempo.
- **Un trámite nuevo no debe requerir programación.** El administrador da de alta un servicio y define sus campos; las solicitudes ya registradas conservan los campos con que se capturaron.
- **Una solicitud la atiende un solo responsable.** Si dos responsables la toman a la vez, solo uno debe quedarse con ella.
- **Cada cambio de estado deja evidencia que no se puede alterar**, con autor, estado anterior, estado nuevo, observación y fecha.
- **Cada servicio tiene un plazo en días hábiles.** Al vencer, la solicitud cambia de estado sola y el área recibe aviso.
- **Cada actor ve y hace solo lo que le corresponde.** Un estudiante no ve la bandeja de un área; un responsable solo atiende las de su área.
- **El estudiante y el área se enteran por correo** de cada cambio, pero si el correo falla la solicitud no se pierde.

Alcance del prototipo

El prototipo implementa los flujos de registro, toma, cambio de estado, cancelación, vencimiento y aviso por correo como API REST. Corre en contenedores locales con PostgreSQL, Redis y Mailpit. La administración se limita a las operaciones descritas en el Ejercicio 2; no es un sistema institucional terminado.

Quedan fuera las pantallas, el despliegue en Render, el correo institucional real, los pagos, la firma electrónica y la integración con el sistema escolar. Los tres primeros son decisiones del prototipo, explicadas en la sección de cambios; los tres últimos ya estaban fuera del caso desde la Evidencia 3.

Arquitectura

> SIGSU es un **monolito modular en capas**. Un solo artefacto Spring Boot contiene cinco módulos: Seguridad, Catálogo, Solicitudes, Notificaciones y Administración. Cada módulo se organiza en capas, y un módulo solo usa a otro a través de su interfaz pública.

La elección responde al alcance del caso. Para dimensionar el diseño se supusieron 8 000 estudiantes y dos trámites al mes por estudiante, unas 16 000 solicitudes mensuales. Es una estimación, no una medición de carga. Un monolito evita agregar red y transacciones distribuidas al prototipo. El registro y su historial se confirman en una misma transacción de PostgreSQL. La capacidad y el escalamiento deberán validarse con pruebas de carga antes de un uso institucional.

Que el monolito sea modular protege la mantenibilidad: Solicitudes no conoce cómo guarda Catálogo sus formularios y a Notificaciones solo le publica un evento. Así, cambiar el envío de correo o la forma de validar un formulario no toca el registro de solicitudes.

Componente	Responsabilidad	Requerimiento que atiende
Seguridad	Autentica cada petición y revisa el rol antes de entrar a cualquier módulo; entrega al módulo el usuario autenticado.	Cada actor ve y hace solo lo que le corresponde.
Catálogo	Guarda áreas, categorías, servicios, versiones de formulario y días inhábiles; ofrece a Solicitudes el esquema vigente, el plazo y el cálculo de días hábiles.	Un trámite nuevo no requiere programación; plazos en días hábiles.
Solicitudes	Registra, toma, cambia de estado, cancela y vence solicitudes; escribe el historial en la misma transacción que cada cambio.	Sin duplicados, un solo responsable, evidencia inalterable, vencimientos.
Notificaciones	Recibe el evento de cambio después del COMMIT y envía el correo en segundo plano.	Aviso por correo sin poner en riesgo la solicitud.
Administración	Gestiona usuarios, roles y la asignación de responsables a áreas.	Cada actor ve y hace solo lo que le corresponde.
PostgreSQL	Garantiza con índices, llaves y restricciones las reglas que no pueden fallar.	Sin duplicados, evidencia inalterable, versiones de formulario.

Tabla 2. Componentes, responsabilidades y requerimientos.

La última fila es la decisión de diseño más importante del equipo: **las reglas que no pueden fallar las garantiza la base de datos, no una consulta previa de la aplicación.** Dos peticiones simultáneas pasarían la misma consulta "¿ya existe una solicitud abierta?" y ambas registrarían. Un índice único parcial no tiene esa ventana.

Herramientas de modelado

> Los anexos A1 a A5 presentan las vistas actualizadas al prototipo.

Herramienta	Para qué la usamos
PlantUML	Borradores de los cinco diagramas mientras discutíamos la arquitectura. Como el diagrama es texto, cada propuesta de cambio se revisaba como un cambio de texto.
Python / ReportLab	Cinco vistas vectoriales actualizadas desde el código y la trazabilidad del prototipo. Se incluyen como diagramas en los anexos del reporte.
Swagger (springdoc)	Modelo de la API generado desde el código: endpoints, cuerpos y respuestas. Permite comparar lo que pide el diagrama de secuencia con lo que expone el prototipo.
Postman	Pruebas de los endpoints. Cada petición de la colección comprueba una regla del modelo.
Git y GitHub	Historial del código, las pruebas y sus evidencias en el repositorio.

Tabla 3. Herramientas y propósito.

Los modelos son útiles cuando sus nombres y relaciones se pueden contrastar con el código. El avance describe los diagramas originales y la matriz recupera sus decisiones; los anexos muestran la versión actualizada al prototipo. No se afirma que esos anexos ya estuvieran publicados en GitHub. Los cambios de autenticación, archivos y despliegue se explican por separado para distinguir el diseño previsto de lo que se construyó.

Diagramas generados

Diagrama	Qué representa
Componentes	Los cinco módulos dentro del artefacto, las interfaces que ofrece y usa cada uno, el evento hacia Notificaciones y los sistemas externos: PostgreSQL, el servidor de correo y el disco de adjuntos.
Despliegue	Contenedores de app, PostgreSQL, Redis y Mailpit, con volúmenes de datos y adjuntos.
Paquetes	Los cinco módulos, el paquete compartido y las dependencias públicas verificadas en el código.
Clases	El modelo de datos dibujado dentro de sus paquetes, con las restricciones que garantiza la base de datos: unicidad, inmutabilidad y solo inserción.
Secuencia	El registro de una solicitud de principio a fin: validación, cálculo del vencimiento, transacción, duplicado detectado por la base de datos y aviso después del COMMIT.

Tabla 4. Diagramas y lo que representa cada uno.

Cómo apoyaron los modelos el desarrollo

- **Planeación.** La secuencia original permitió discutir validación, transacción, historial y aviso posterior al COMMIT. En el prototipo el registro recibe JSON y los adjuntos se suben después. El anexo A4 representa ese flujo final y la sección de cambios explica por qué se separaron ambas operaciones.
- **Comunicación.** Como los cinco diagramas comparten nombres, el equipo pudo repartirse el trabajo por paquete y hablar de `RegistrarSolicitud` o de `ICatalogo` sabiendo todos a qué se refería.
- **Revisión.** El diagrama de clases hizo visible un error de la primera versión: el historial estaba dibujado como composición de la solicitud, lo que implica que se borra con ella. Lo cambiamos por una referencia, porque el historial es evidencia y debe sobrevivir aunque la solicitud se archive.
- **Modificación.** La vista de despliegue final sustituye Render y Microsoft 365 por Compose y Mailpit. El anexo A5 muestra esa configuración local.

Ejercicio 1: análisis y modelos

> El primer ejercicio recupera el caso, los actores, la arquitectura y las decisiones previas. La matriz siguiente expresa cómo se planeó comprobarlas; el Ejercicio 3 presenta los resultados y los anexos muestran las vistas finales.

Matriz inicial de trazabilidad

Antes de implementar, cada requerimiento quedó ligado a una decisión, a un elemento del modelo y a la forma en que pensábamos comprobarlo. Esta es la versión previa a la implementación; la versión final, contrastada con el prototipo, está en la sección del Ejercicio 3.

Requerimiento	Decisión de diseño	Dónde quedó en el modelo	Cómo se comprobaría
Sin dos solicitudes abiertas del mismo servicio	Índice único parcial sobre (estudiante, servicio) para los estados abiertos; sin consulta previa	Clases: nota en `Solicitud`. Secuencia: pasos 11 a 15	Registrar dos veces el mismo servicio: la segunda petición responde 409 con el folio existente
Trámites nuevos sin programación	Formulario de cada versión como JSON Schema en una fila inmutable; la solicitud apunta a su versión	Clases: `FormularioServicio {inmutable}` y llave compuesta desde `Solicitud`	Datos que no cumplen el esquema responden 422; una versión nueva no cambia las solicitudes viejas
Un solo responsable por solicitud	Bloqueo optimista: la toma es un `UPDATE` condicionado a la versión leída	Clases: `Solicitud.version` y `tomar()`. Tabla de estados	Dos tomas simultáneas: una responde 200 y la otra 409

Requerimiento	Decisión de diseño	Dónde quedó en el modelo	Cómo se comprobaría
Evidencia inalterable	`Historial` de solo inserción, escrito en la misma transacción que el cambio y fuera de la composición de `Solicitud`	Clases: `Historial {solo inserción}`	Un `UPDATE` sobre el historial es rechazado por la base de datos
Plazos en días hábiles y vencimiento automático	`venceEn` calculado al registrar con el plazo del servicio y `DiaInhabil`; proceso programado cada 15 minutos	Clases: `calcularVencimiento()`, `DiaInhabil`. Componentes y despliegue: proceso de vencimientos	La respuesta del registro trae un `venceEn` que salta fines de semana e inhábiles; una solicitud vencida cambia de estado sola
Cada actor solo lo suyo	Seguridad revisa el rol antes de cualquier módulo; usuario con varios roles y asignación a áreas	Componentes: Seguridad al frente. Clases: `Usuario`, `Rol`, `ResponsableArea`	Un estudiante que pide la bandeja recibe 403
Aviso sin perder la solicitud	Evento `CambioDeSolicitud` atendido después del COMMIT, en segundo plano	Componentes: evento hacia Notificaciones. Secuencia: pasos 17 a 20	El correo llega después del registro; si el servidor de correo está apagado, la solicitud se registra igual
Módulos sustituibles	Dependencias entre módulos solo hacia interfaces de aplicación	Paquetes: tres dependencias, sin ciclos	Solicitudes solo importa la interfaz `ICatalogo`, no clases internas de Catálogo

Tabla 5. Matriz inicial de trazabilidad.

Decisiones tomadas antes de implementar

- **Usuario con varios roles en lugar de herencia por rol.** Un profesor puede ser estudiante de posgrado y responsable de laboratorio con la misma cuenta. Con herencia necesitaría dos usuarios.
- **Datos de la solicitud en JSONB validados con JSON Schema, no en una tabla atributo-valor.** Una tabla atributo-valor obliga a reconstruir cada solicitud con muchas filas y no puede validar tipos ni campos obligatorios. JSONB guarda la solicitud completa en una columna, y el esquema define qué es válido.
- **Estados comunes a todos los servicios.** Siete estados describen cualquier trámite. Estados por servicio habrían convertido cada trámite nuevo en un cambio de código, que es lo que el catálogo debe evitar.
- **Sin cola de mensajes para el correo.** El estado vive en SIGSU y el correo solo avisa. Si un correo se pierde, la solicitud no se pierde, así que una cola o una bandeja de salida no se justifican con esta carga.

El Ejercicio 2 continúa en la página siguiente con el flujo implementado y sus resultados.

Ejercicio 2: implementación

El prototipo permite recorrer un trámite mediante una API REST. El estudiante consulta el catálogo, revisa el formulario del servicio y registra sus datos. Recibe un folio, el estado REGISTRADA y la fecha de vencimiento. Después consulta el historial, sube adjuntos o cancela mientras nadie lo atiende. El responsable revisa la bandeja de su área, toma la solicitud y la cierra como ATENDIDA o RECHAZADA.

Spring Boot y organización del código

Java 21 y Spring Boot 4.1.1 reúnen los cinco módulos del diseño en un solo artefacto. Cada módulo utiliza las capas controller, service, model, repository y dto donde las necesita. Los controladores reciben HTTP, los servicios ejecutan los casos de uso y los repositorios acceden a PostgreSQL. Solicitudes consulta a Catálogo mediante ICatalogo y a Seguridad mediante IUsuarios; Notificaciones recibe el evento CambioDeSolicitud.

RegistrarSolicitud obtiene las condiciones del servicio y valida sus datos contra el formulario vigente. Dentro de la transacción guarda Solicitud, su primer Historial y publica el evento de cambio. Si el formulario es inválido responde 422. Si el índice de solicitudes abiertas detecta duplicado, la transacción se revierte y la API responde 409 con el folio existente cuando todavía puede localizarlo. Un registro válido responde 201.

Persistencia y reglas del modelo

PostgreSQL 16 conserva usuarios, servicios, versiones de formulario, solicitudes, historial y metadatos de adjuntos. Flyway crea el esquema con V1__esquema_inicial.sql y carga usuarios y catálogo de prueba con V2__catalogo_y_usuarios_de_prueba.sql. Hibernate usa ddl-auto: validate; no modifica por su cuenta las restricciones del esquema.

Los datos se guardan en JSONB y cada solicitud conserva una referencia compuesta a la versión del formulario que utilizó. El índice parcial solicitud_abierta impide duplicados para los cuatro estados abiertos. La toma utiliza un UPDATE condicionado a la versión leída; si otra persona la tomó primero devuelve 409. El historial se escribe en la misma transacción y sus triggers rechazan UPDATE, DELETE y TRUNCATE.

Los archivos se cargan después por POST /solicitudes/{folio}/adjuntos, con límite de 5 MB por archivo. El archivo vive en un volumen y su metadato en PostgreSQL. Si falla el guardado del metadato se intenta eliminar el archivo recién escrito. Un corte abrupto aún puede dejar un archivo huérfano; ese límite permanece fuera de la recuperación del prototipo.

Fuentes de implementación: RegistrarSolicitud.java; migración V1. Las vistas A1, A3 y A4 ubican estos elementos.

Seguridad, ejecución y pruebas

Spring Security autentica con HTTP Basic contra la tabla usuario y compara contraseñas BCrypt. La configuración exige rol por ruta. Los servicios comprueban además la propiedad de la solicitud y la pertenencia al área; tener credenciales válidas no permite atender cualquier trámite. Swagger describe los endpoints y permite explorarlos sin una interfaz Angular.

El filtro de idempotencia se ejecuta dentro de Spring Security después de autenticar y autorizar. Cuando un POST incluye Idempotency-Key, Redis conserva la respuesta durante 24 horas. Repetir la misma petición devuelve el mismo cuerpo y estado. Una llave en procesamiento responde 409 y reutilizarla con otro cuerpo responde 422. La idempotencia reconoce reintentos; el índice de PostgreSQL protege la regla de no tener dos trámites abiertos equivalentes.

Compose reúne la app, PostgreSQL, Redis y Mailpit. Los volúmenes conservan datos y adjuntos al reiniciar los contenedores. El proceso @Scheduled revisa vencimientos cada 15 minutos y registra el autor de sistema como autorId: null. AvisarCambio recibe el evento AFTER_COMMIT y envía el correo con @Async. Mailpit permite observar el correo sin credenciales institucionales. Un fallo de SMTP no revierte lo confirmado, pero no hay cola persistente para reintentar el envío.

Resultados conservados del prototipo

La colección Postman se ejecutó con Newman dos veces sobre los mismos servicios, sin reiniciarlos entre corridas. Los archivos versionados contienen 48 peticiones y 101 aserciones por corrida, sin fallos. El registro Maven contiene 18 pruebas unitarias sin fallos ni errores. Los anexos A6 a A8 seleccionan esos resultados y las pruebas directas de concurrencia, SQL y vencimientos. Este reporte analiza los registros guardados; no los presenta como una ejecución nueva.

Las respuestas HTTP prueban comportamiento observable, pero no bastan para comprobar la estructura interna. Por eso el equipo conserva además la definición del índice, los rechazos de SQL, la carrera con dos responsables y el historial producido por el scheduler. Una prueba exitosa del correo confirma su recepción en Mailpit; el orden AFTER_COMMIT se comprueba también en el código.

Git/GitHub y alcance real

Git conserva cambios separados de esquema, implementación, pruebas, evidencias, trazabilidad y diferencias de diseño. GitHub publica el repositorio sigsu-service. La versión utilizada para este reporte es faa4156a, cuyo mensaje es "Documentar las diferencias entre el diseño de la Evidencia 3 y el prototipo". El historial consultado contiene 13 commits. Esto identifica la versión analizada sin atribuir trabajo individual que el historial no demuestre.

La administración permite crear usuarios con roles, asignar responsables, crear servicios, publicar formularios y ajustar plazos, días inhábiles y parámetros. No ofrece alta o edición de áreas y categorías, cambio de roles de usuarios existentes ni desactivación de usuarios. Los parámetros guardados no reconfiguran la infraestructura en caliente. Son límites explícitos, no funcionalidades comprobadas.

Ejercicio 3: correspondencia entre modelo y prototipo

La matriz inicial expresaba qué se pensaba comprobar. Esta matriz final contrasta siete decisiones con el código y los resultados guardados. Los diagramas A1 a A5 son vistas actualizadas para esta entrega; no sustituyen la explicación de las diferencias respecto del diseño previo.

Elemento del modelo	Herramienta	Diagrama	Implementación	Tecnología	Prueba / resultado	Evidencia
Solicitud sin duplicados abiertos	Python/ReportLab; Postman/Newman; psql	Clases A3; secuencia A4	RegistrarSolicitud; índice solicitud_abierta en V1	Spring MVC; PostgreSQL	R2: duplicado abierto 409 con folio. INSERT directo rechazado por el índice.	newman-resultado.json; indice-parcial.txt. A6.
Formulario inmutable por versión	Python/ReportLab; Postman/Newman	Clases A3	FormularioServicio.validar; FK (servicio, versión)	JSON Schema; JSONB; JPA	R1: falta motivo 422. R8: solicitud previa conserva v1; nueva usa v2.	newman-resultado.json, requests R1 y R8. A6.
Un responsable por solicitud	Python/ReportLab; HTTP concurrente	Clases A3	TomarSolicitud; UPDATE con versión en SolicitudRepositorio	PostgreSQL; control optimista	Dos POST con versión 0: un 200 y un 409. Ganadora EN_ATENCION, versión 1.	toma-concurrente.txt. A7.
Historial de solo inserción	Python/ReportLab; psql	Clases A3; secuencia A4	RegistrarCambio; triggers de historial en V1	Transacción JPA; triggers SQL	UPDATE, DELETE y TRUNCATE rechazados: historial es de solo inserción.	historial-inmutable.txt. A7.
Roles y asignación a áreas	Python/ReportLab; Postman/Newman	Componentes A1; clases A3	ConfiguracionSeguridad; comprobaciones de propiedad y área	Spring Security; HTTP Basic	R7: estudiante pide bandeja 403. R3: otra área no puede tomar 403.	newman-resultado.json, requests R7 y R3.
Vencimiento automático	Python/ReportLab; SQL y logs	Componentes A1; despliegue A5	VencerSolicitudes; Solicitud.vencer; historial con autor null	@Scheduled; PostgreSQL	Cron temporal de 1 minuto: pasa a SIN_RESPUESTA sin endpoint manual; vuelve al cron normal.	vencimiento-programado.txt; restauracion-compose.txt. A8.
Aviso después del COMMIT	Python/ReportLab; Postman; Mailpit	Componentes A1; secuencia A4	AvisarCambio recibe CambioDeSolicitud	AFTER_COMMIT; @Async; SMTP	R6: Mailpit contiene correo del folio. El código confirma el orden posterior al commit.	newman-resultado.json; mailpit.txt; AvisarCambio.java. A8.

Tabla 6. Matriz modelo–prototipo. Los archivos de evidencia son relativos a docs/evidencias/, excepto el código. La relación detallada de elementos originales está en TRAZABILIDAD.md. Las pruebas HTTP acreditan respuestas; SQL, código y logs acreditan restricciones y comportamiento interno.

Cambios entre diseño e implementación

Se mantuvieron el monolito modular, el índice parcial de duplicados, los formularios por versión, la toma con versión, el historial transaccional y el aviso posterior al COMMIT. La tabla resume los ajustes documentados por Walter en CAMBIOS.md. El anexo de despliegue representa el prototipo local, no la infraestructura prevista en Render.

Diseño previo	Implementación final	Justificación e impacto
Pantallas generadas en servidor; sesión con cookie HttpOnly.	API REST y Swagger; HTTP Basic sin sesión contra usuario.	Permite probar reglas con Postman y conservar control por roles. No hay Angular, pantallas ni autenticación institucional.
Capas web, aplicación, dominio e infraestructura. Solicitudes usa ICatalogo.	controller, service, model, repository y dto; además usa IUsuarios.	Convención de Spring y consulta del correo por interfaz pública. No se importan entidades internas del otro módulo.
Historial protegido por permisos INSERT-only.	Triggers rechazan UPDATE, DELETE y TRUNCATE; formularios también se protegen.	La base aplica la inmutabilidad en la ejecución normal, incluso a operaciones del propietario. Un administrador aún puede cambiar el esquema o desactivar triggers.
Registro de datos y archivos en una sola secuencia.	Registro JSON primero; carga multipart de adjuntos después.	Simplifica validación e idempotencia y evita escribir archivos para un registro duplicado. Un corte puede dejar archivos huérfanos.
Render Web Service, Postgres, disco persistente y Cron Job. Correo Microsoft 365.	Compose local con app, PostgreSQL, Redis, Mailpit y volúmenes. @Scheduled cada 15 minutos.	Se reproduce sin cuentas de nube ni credenciales de correo. El cron procesa filas con SKIP LOCKED y genera historial y evento por fila.
No había filtro de idempotencia ni Redis.	Filtro tras autorización; llave por usuario, huella y respuesta almacenada con TTL.	Un reintento recibe su respuesta original. Complementa el índice parcial, que protege solicitudes distintas. Redis caído afecta las peticiones con llave.
Mapeo de persistencia sin mecanismo completo de migraciones.	Flyway V1/V2; Hibernate valida el esquema.	SQL explícito conserva índice parcial, JSONB, FK compuestas y triggers que el mapeo de entidades no genera por completo.
Administración de usuarios, roles, áreas, categorías y servicios.	Operaciones mínimas de servicios, formularios, usuarios, asignaciones, calendario y parámetros.	Se priorizan los flujos demostrables. Áreas/categorías se cargan con semilla. No se cambian roles existentes ni se desactivan usuarios por API.
Tomar fija EN_ATENCION; el diseño no precisaba tomar una SIN_RESPUESTA.	REGISTRADA pasa a EN_ATENCION; SIN_RESPUESTA pasa a DEMORADO.	Conserva la condición de retraso. Los vencimientos usan autorId null para representar sistema sin crear un usuario artificial.

Tabla 7. Cambios seleccionados. El documento del repositorio contiene 17 diferencias, incluidas las precisiones sobre folio y hora de vencimiento. Esta selección explica los cambios que afectan la correspondencia entre modelo, herramientas y prototipo.

Análisis técnico del equipo

La decisión más útil fue convertir cada problema del caso en una regla comprobable. "No duplicar" dejó de ser una recomendación de pantalla y se volvió un índice parcial. "Un responsable" se convirtió en una actualización condicionada por versión. "Conservar evidencia" se expresó con una entidad de historial, una transacción compartida y triggers. Así se pudo comprobar qué correspondencia existe entre el modelo y la implementación.

Las pruebas conservadas respaldan esos puntos con distintos tipos de evidencia. La carrera de toma tiene una ganadora y un conflicto; SQL rechaza cambios al historial; el scheduler deja un movimiento con autor de sistema. Las dos corridas de Newman muestran que la colección se puede repetir sin reiniciar los servicios. Los conteos no prueban ausencia de errores en todos los escenarios, rendimiento institucional ni recuperación ante pérdida del servidor.

La modularidad ayudó a ubicar las responsabilidades. Catálogo conoce el esquema y el plazo; Solicitudes controla el trámite; Seguridad identifica al actor; Notificaciones recibe el cambio confirmado. La trazabilidad permite revisar esos límites en imports, interfaces y archivos, además de revisar las respuestas HTTP. Un diagrama solo aporta si sus relaciones se pueden seguir hasta esos elementos concretos.

También hubo compromisos. Basic y Compose facilitaron la demostración local, pero quedan pendientes la interfaz y la operación institucional. El correo posterior al commit evita revertir el trámite, aunque puede perderse si el proceso cae antes de enviarlo. Los adjuntos no tienen recuperación de huérfanos ante un corte abrupto. Redis agrega idempotencia y también una dependencia operativa. Son límites que el modelo y el reporte deben hacer visibles.

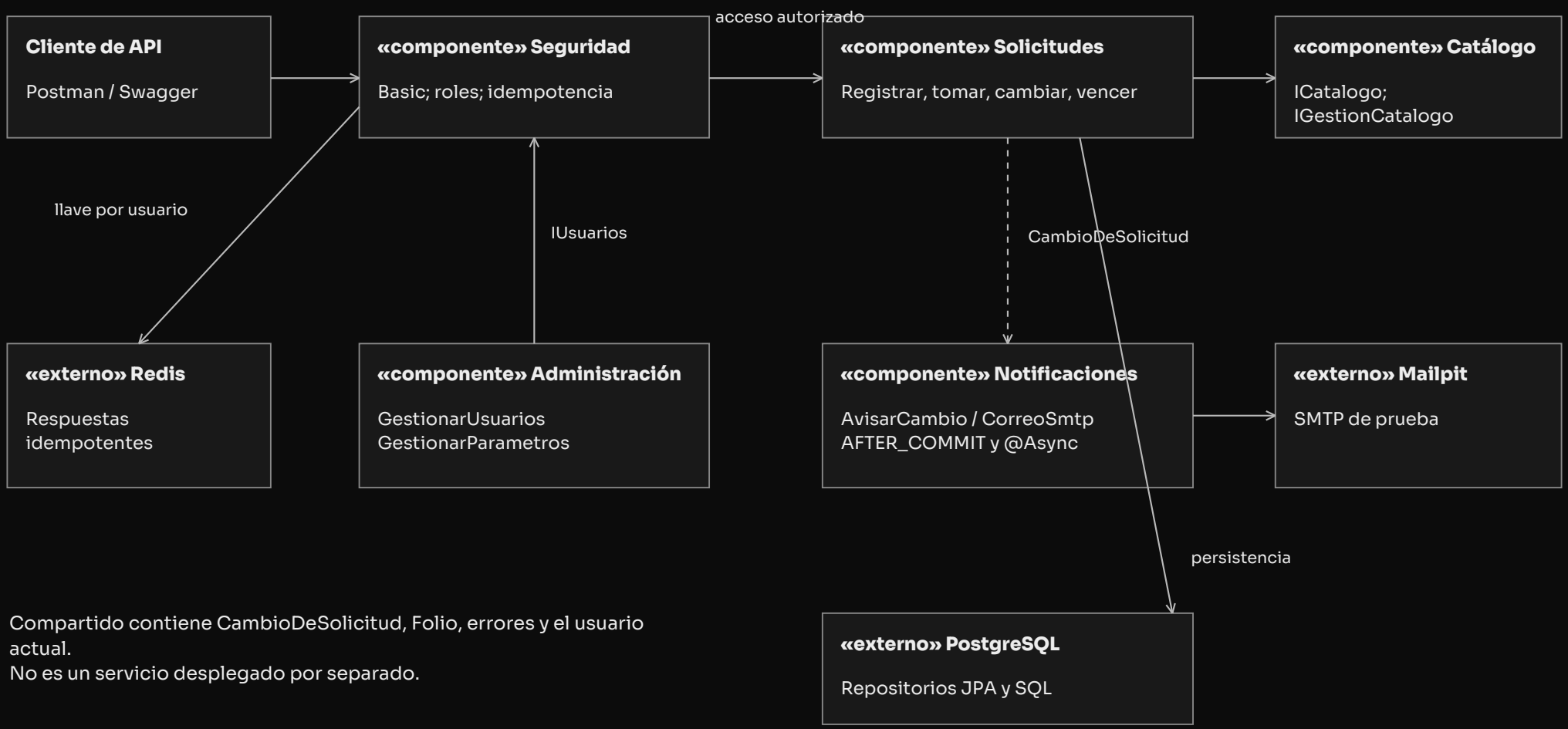
Conclusiones

El prototipo demuestra los flujos principales del caso y las reglas seleccionadas del modelo mediante código y resultados guardados. La cadena caso, arquitectura, modelado, herramienta, implementación, prueba, evidencia y análisis se puede seguir en las matrices y los anexos. No basta conservar nombres de clases ni agregar capturas sin explicación; cada evidencia debe responder qué decisión técnica verifica.

El monolito modular fue suficiente para construir una demostración con una sola base transaccional y responsabilidades separadas. Las diferencias entre el diseño y la implementación quedaron justificadas, en especial la API REST, los adjuntos posteriores al registro, la infraestructura local y el filtro de idempotencia. El siguiente paso hacia uso real requiere validar carga, recuperación, seguridad de despliegue y entrega confiable de avisos; esos aspectos no se dan por probados en esta evidencia.

A1. Componentes del prototipo

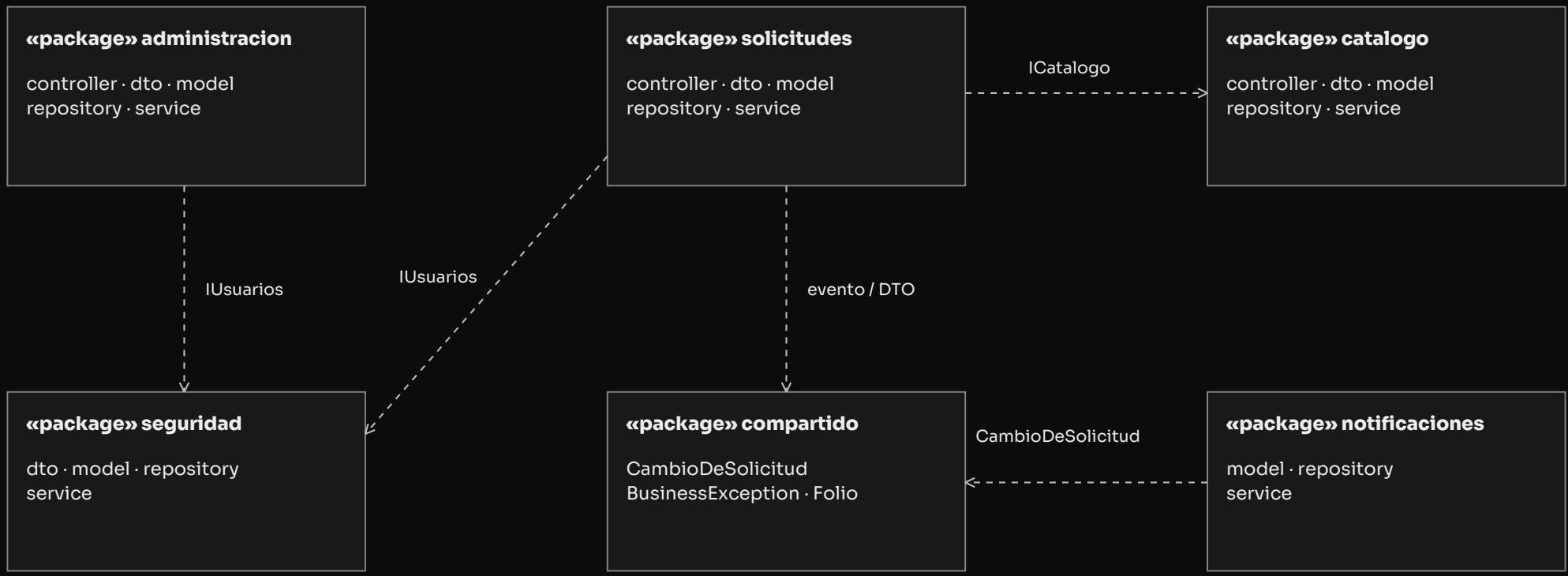
Vista actualizada. Un artefacto Spring Boot, cinco módulos y un paquete compartido. Las flechas nombran los contratos principales.



Compartido contiene CambioDeSolicitud, Folio, errores y el usuario actual.
No es un servicio desplegado por separado.

A2. Paquetes y dependencias públicas

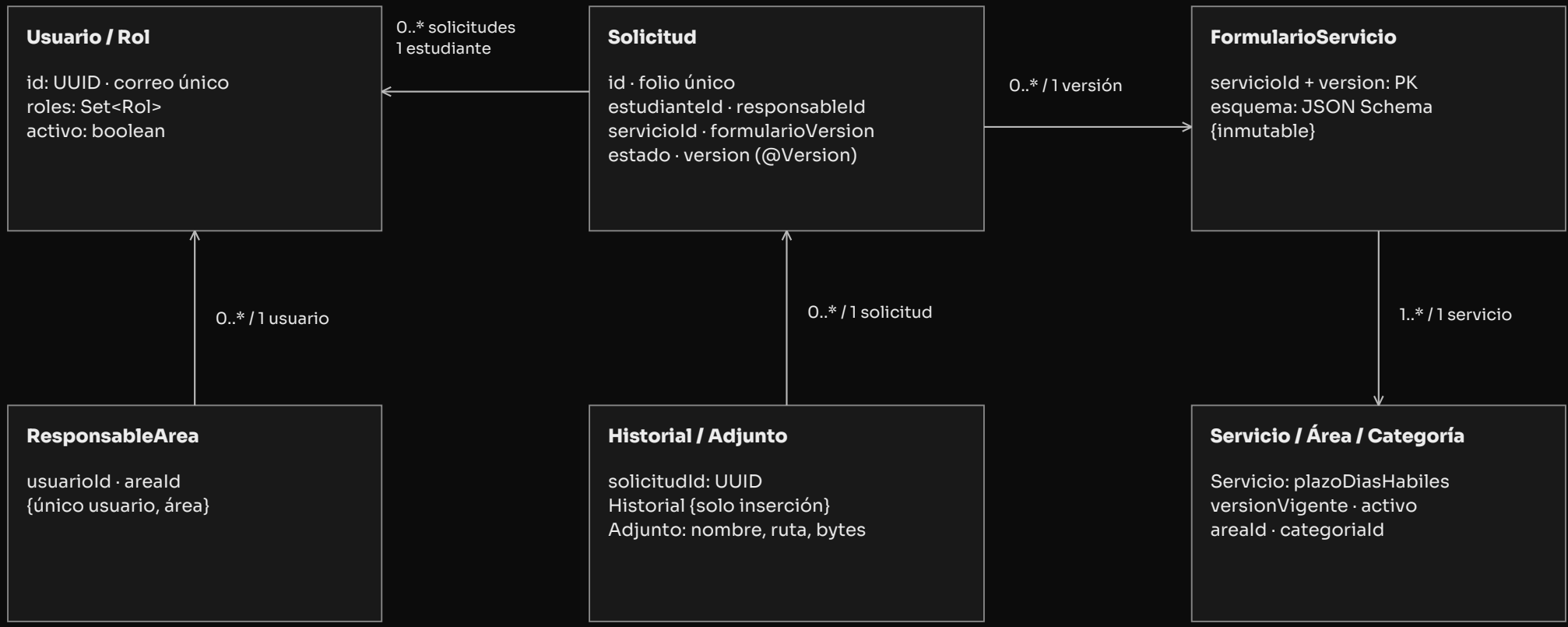
Vista reducida de mx.utch.sigsu. Se omiten imports de utilidades. Una flecha indica dependencia de código, no una llamada de red.



Los repositorios de Catálogo y Seguridad no son importados por Solicitudes. La administración del catálogo se expone desde CatalogoController; la ruta administrativa no obliga a crear una dependencia inversa.

A3. Clases y relaciones persistidas

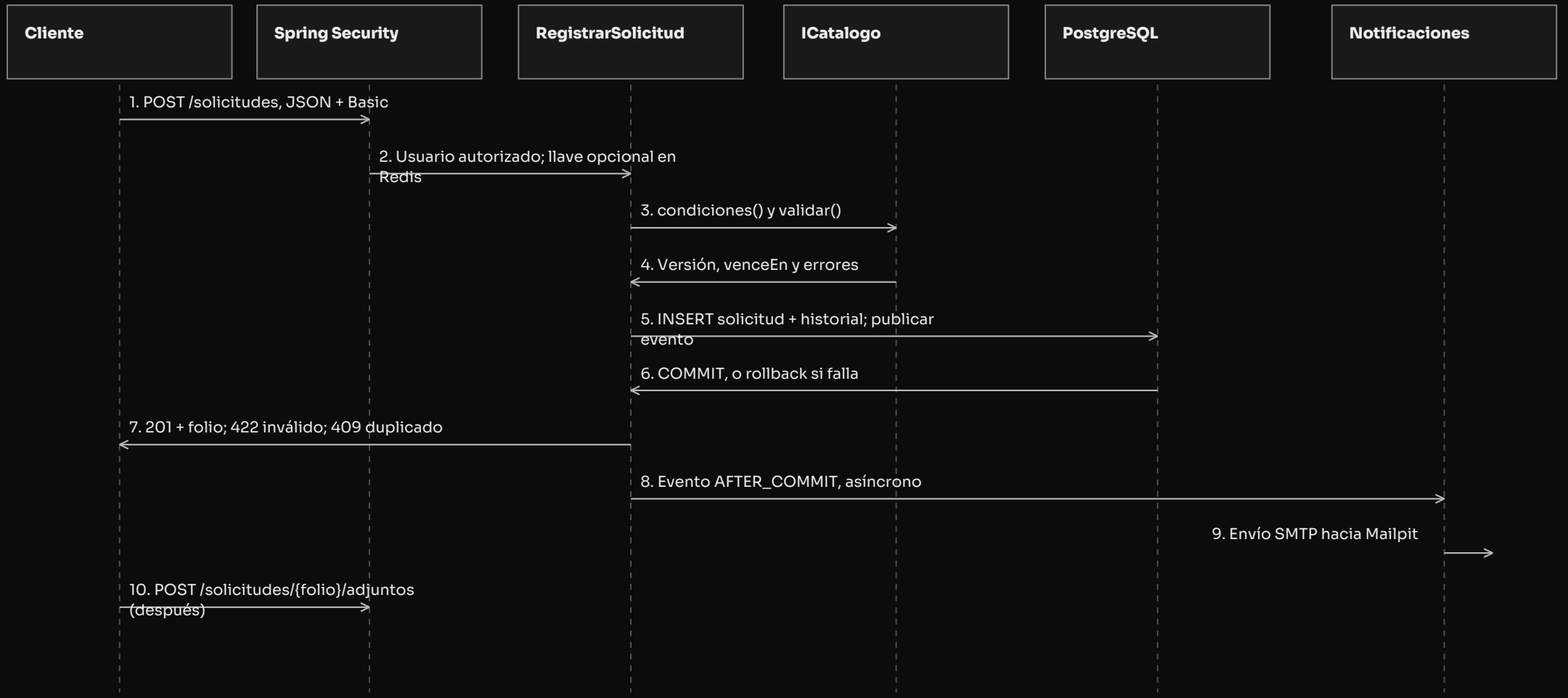
Vista simplificada del modelo JPA y las FK de V1. Las asociaciones representan referencias por UUID; no implican @ManyToOne en Java.



Solicitud conserva la versión del formulario por FK compuesta. solicitud_abierta es único por estudiante y servicio solo en estados abiertos. Historial tiene autor opcional para el sistema. DiaInhabil y Parametro no se dibujan por espacio.

A4. Secuencia de registro

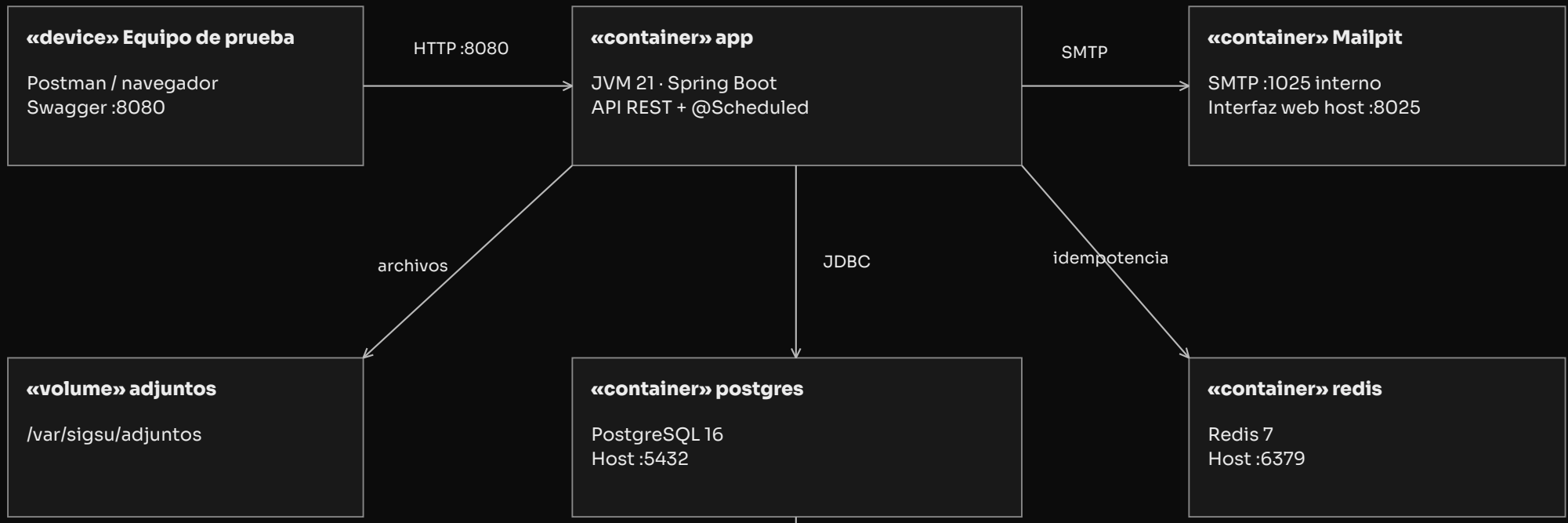
Flujo implementado. Los adjuntos se cargan en una petición posterior. La validación y el registro usan la transacción de RegistrarSolicitud.



El envío puede terminar antes o después de que el cliente reciba el 201; siempre se agenda tras confirmar. Mailpit acredita recepción, mientras la anotación AFTER_COMMIT se comprueba en AvisarCambio.java.

A5. Despliegue local con Compose

Configuración efectiva de compose.yaml. No representa Render ni una instalación institucional. Puertos predeterminados publicados al host.



La misma imagen ejecuta el proceso de vencimientos.
Los volúmenes sobreviven a un reinicio normal.

A6. Pruebas repetibles y respuestas

Las imágenes siguientes son capturas documentales de los registros versionados del prototipo. No son capturas de la interfaz de Postman ni ejecuciones nuevas. El commit analizado es faa4156a. Cada pie identifica el archivo original y la decisión que permite comprobar.

Extracto literal · newman-salida.txt

```
┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│                requests │                 48 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│            test-scripts │                 48 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│      prerequest-scripts │                  2 │                  0 │
├─────────────────────────┼────────────────────┼────────────────────┤
│              assertions │                101 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 5.7s                                          │
├───────────────────────────────────────────────────────────────────┤
│ total data received: 18.57kB (approx)                             │
├───────────────────────────────────────────────────────────────────┤
│ average response time: 102ms [min: 4ms, max: 663ms, s.d.: 87ms]   │
└───────────────────────────────────────────────────────────────────┘
```

Figura A6.1. Primera corrida de Newman. El resumen registra 48 peticiones y 101 aserciones, sin fallos. La segunda corrida conserva los mismos conteos sin reiniciar servicios. Los JSON de ambas corridas contienen run.failures vacío. Fuente: primera salida y segunda corrida.

Extracto literal · pruebas-unitarias.txt

```
[INFO] Tests run: 18, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

Figura A6.2. Resumen Maven de 18 pruebas sin fallos ni errores. Prueba reglas puntuales del modelo y del filtro; no sustituye la comprobación HTTP. Fuente: pruebas-unitarias.txt.

Peticiones seleccionadas de la matriz

En newman-resultado.json, R2 devuelve 409 ante duplicado abierto; R1 devuelve 422 si falta motivo; R8 conserva formularioVersion 1 en una solicitud previa y usa 2 en una nueva. R7 devuelve 403 al estudiante que solicita la bandeja. La colección incluye las aserciones que comprueban cada resultado.

A7. Concurrencia e inmutabilidad

toma-concurrente.txt · cuerpo ganador abreviado

```
Dos POST /solicitudes/SIGSU-2026-000019/tomar concurrentes, ambos con version=0.
Responsable 1: status 409
{"status":409,"mensaje":"otro responsable la tomó primero o el estado no permite
tomar","folioExistente":null,"errores":[]}
Responsable 2: status 200
Extracto JSON de la ganadora:
{"folio": "SIGSU-2026-000019", "estado": "EN_ATENCION", "version": 1}
```

Figura A7.1. Dos responsables intentan tomar el mismo folio con versión 0. Uno recibe 409 y el otro 200. El cuerpo ganador queda en EN_ATENCION con versión 1. Esto demuestra la exclusión en una carrera real, además del caso secuencial de Postman. Fuente: toma-concurrente.txt.

Extracto literal · historial-inmutable.txt

```
psql -c "UPDATE historial SET observacion='alterado'"
ERROR:  historial es de solo inserción
psql -c "DELETE FROM historial"
ERROR:  historial es de solo inserción
psql -c "TRUNCATE historial"
ERROR:  historial es de solo inserción
psql -c "UPDATE formulario_servicio SET version=version"
ERROR:  formulario_servicio es de solo inserción
```

Figura A7.2. PostgreSQL rechaza UPDATE, DELETE y TRUNCATE de historial y UPDATE del formulario. La restricción está en los triggers de V1; no depende de esconder botones. La evidencia no afirma protección ante un administrador que altere el esquema. Fuente: historial-inmutable.txt.

A8. Vencimiento y aviso por correo

Extracto literal · vencimiento-programado.txt

```
Override SIGSU_VENCIMIENTOS_CRON='0 * * * * *'; SQL de Hibernate habilitado solo durante la prueba.
UPDATE solicitud SET vence_en=now()-interval '1 day' WHERE folio='SIGSU-2026-000020';
UPDATE 1
Transición programada observada tras 23 segundos; sin llamar /administracion/vencimientos.
 SIGSU-2026-000020 | SIN_RESPUESTA |            | REGISTRADA    | Lucía Torres | registro      |
2026-10-29 03:26:37.711478+00
 SIGSU-2026-000020 | SIN_RESPUESTA | REGISTRADA | SIN_RESPUESTA | sistema      | plazo vencido |
2026-10-29 03:27:00.001944+00
```

Figura A8.1. Se fuerza el vencimiento al pasado y se configura temporalmente el cron cada minuto. El registro conserva la transición REGISTRADA a SIN_RESPUESTA por sistema, sin llamar al endpoint manual. Al terminar se restaura la frecuencia normal de 15 minutos. Esta prueba no es una medición de rendimiento. Fuentes: vencimiento programado y restauración.

Extracto literal · mailpit.txt

```
GET /api/v1/messages?limit=1000; total: 40
SIGSU-2026-000020: SIN_RESPUESTA | evidencia.c8157719-fdfe-4faf-a491-e23a3213795f@alumnos.utch.edu.mx,
ti@utch.edu.mx
SIGSU-2026-000020: REGISTRADA | evidencia.c8157719-fdfe-4faf-a491-e23a3213795f@alumnos.utch.edu.mx,
ti@utch.edu.mx
```

Figura A8.2. Mailpit conserva los avisos del mismo folio en REGISTRADA y SIN_RESPUESTA, dirigidos al estudiante y a TI. Esto acredita recepción en el servidor de prueba. El orden posterior al COMMIT se verifica en AvisarCambio.java. Fuente del listado: mailpit.txt.

Referencias y fuentes de evidencia

Equipo 3. (2026). SIGSU, prototipo y documentación. Versión faa4156a807120e89fb75ad80b8c6f8107e4e95f. Repositorio público: github.com/ImWalterGZ/sigsu-service. Consultado el 29 de octubre de 2026.

Equipo 3. (2026). Reporte técnico integrador SIGSU. Avance de Walter del 29 de octubre. Se conserva el caso, la arquitectura y el ejercicio 1; se corrige el alcance y se completa la relación con el prototipo.

Equipo 3. (2026). Trazabilidad del diseño a la implementación y sus pruebas. Fuente detallada de las relaciones modelo, código y petición de Postman.

Equipo 3. (2026). Cambios entre el diseño y el prototipo. Fuente de las decisiones resumidas en la tabla 7.

Equipo 3. (2026). Notas de implementación; verificación local; índice de evidencias. Los registros anexados corresponden a ejecuciones conservadas en el repositorio.

Bass, L., Clements, P., y Kazman, R. (2021). Software architecture in practice, cuarta edición. Addison-Wesley.

Sommerville, I. (2011). Ingeniería de software, novena edición. Pearson Educación.

Object Management Group. (2017). Unified Modeling Language, versión 2.5.1. https://www.omg.org/spec/UML/2.5.1

JSON Schema. Draft 2020-12. https://json-schema.org/specification

PostgreSQL Global Development Group. PostgreSQL 16, índices parciales. https://www.postgresql.org/docs/16/indexes-partial.html

Spring. Spring Boot reference documentation. https://docs.spring.io/spring-boot/

Cómo verificar esta entrega

El repositorio conserva el código, la colección Postman y los registros utilizados. El SHA citado identifica la versión analizada, aunque después cambie la rama principal. Los anexos seleccionan resultados guardados en JSON y texto; no atribuyen pruebas nuevas. La colección y los scripts de verificación permiten repetir las comprobaciones del prototipo.

Los cinco diagramas finales se incluyen en este PDF como vistas actualizadas del prototipo implementado. El contraste con el diseño previo está en la matriz inicial y en la tabla de cambios. Los programas usados para maquetar el reporte y dibujar estas vistas no forman parte del repositorio de la aplicación.